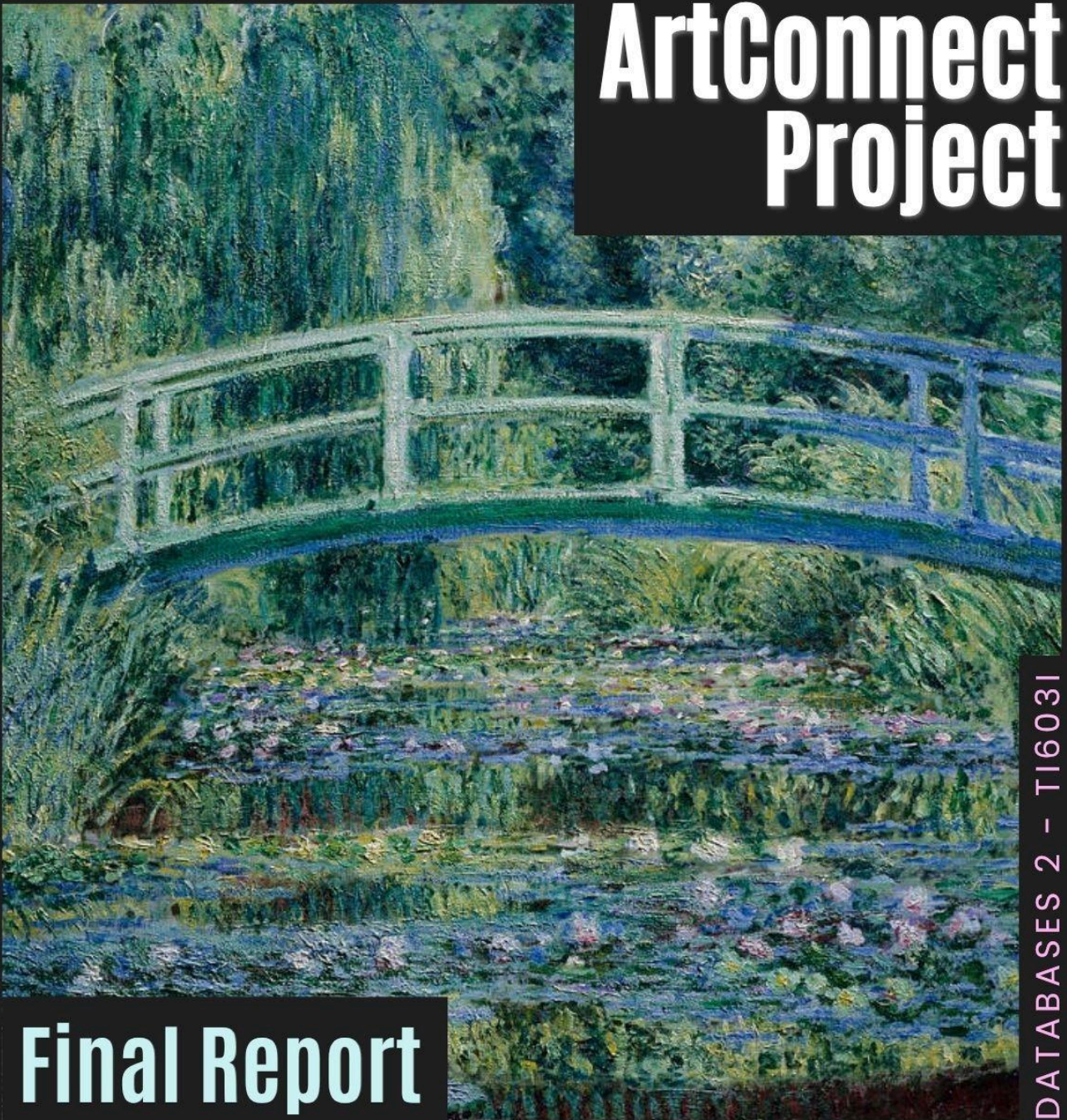
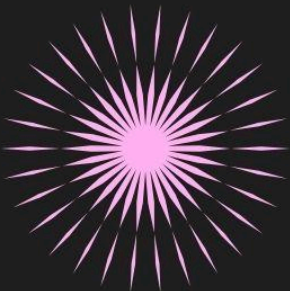


ArtConnect Project



DATABASES 2 - TI603I

Final Report



Oscar MASDUPUY & Meïssa CHINI

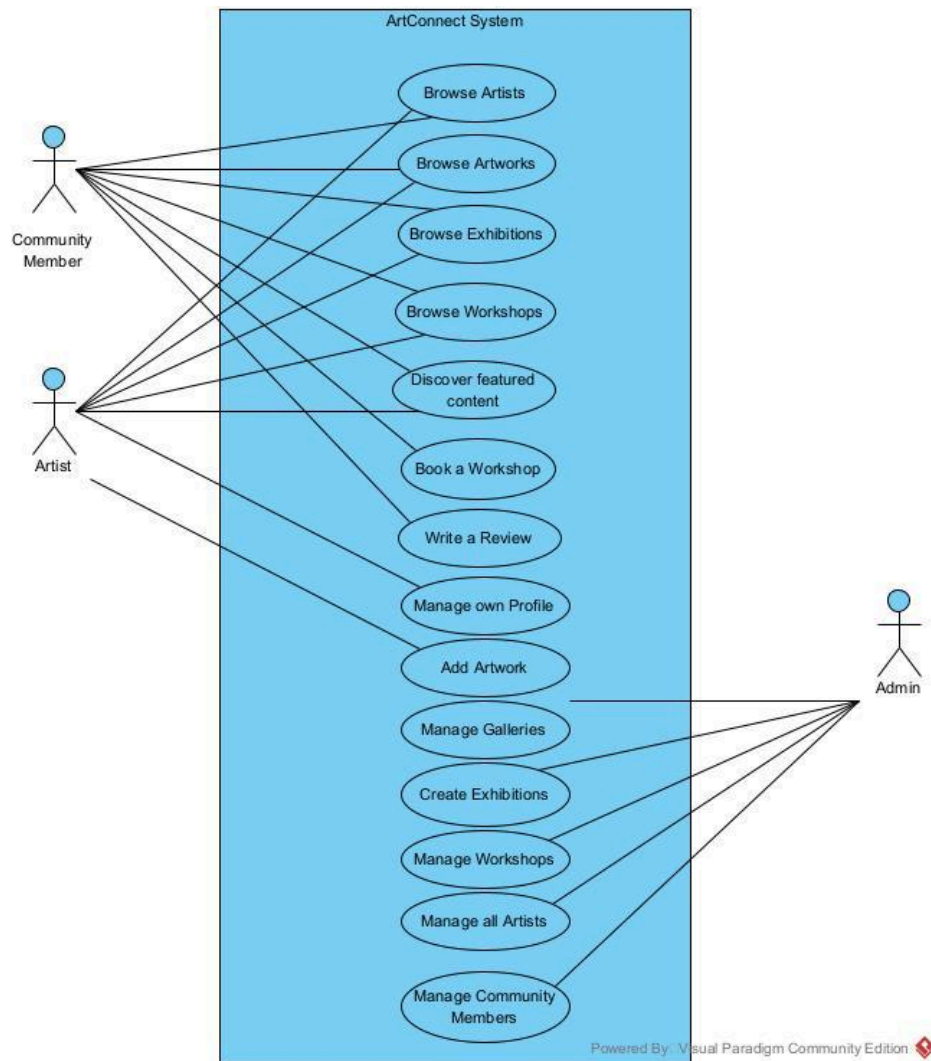
Database Project – Final Report
INGE1 – Int 2

Step 1: Understanding ArtConnect and Defining the Functional Scope

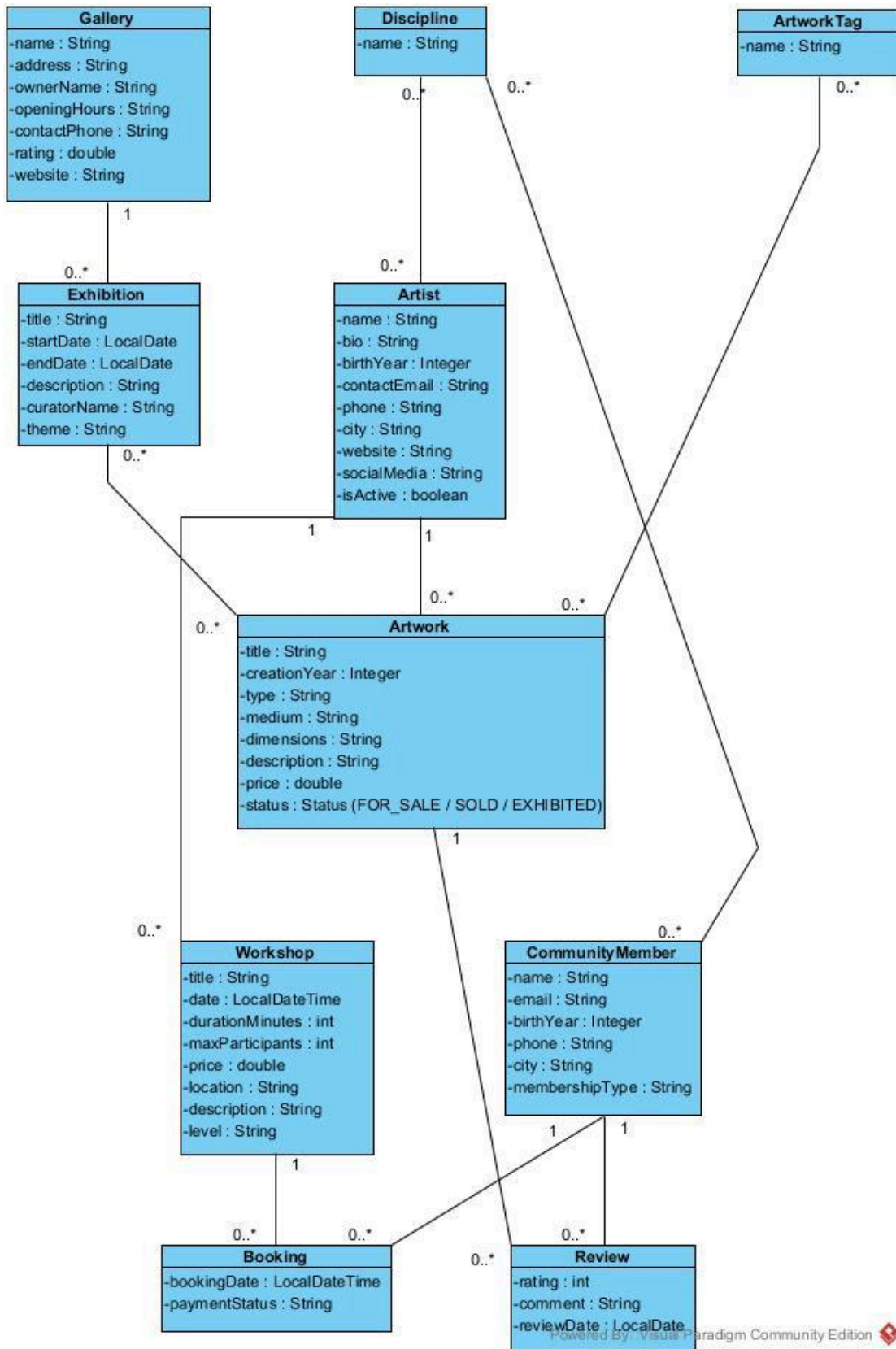
List of features already present in the interface:

- View the complete list of artists with name, city, email, and birth year
- Search artists by name keyword
- Filter artists by discipline (Painting, Sculpture, Photography...)
- View the complete list of artworks with title, year, type, price, and artist name
- View the list of exhibitions with title, dates, gallery name, and theme
- View the list of workshops with title, date, instructor name, and price
- View the list of galleries with name, address, and rating
- View the list of community members
- Discover featured exhibitions and upcoming workshops

Use case Diagram:

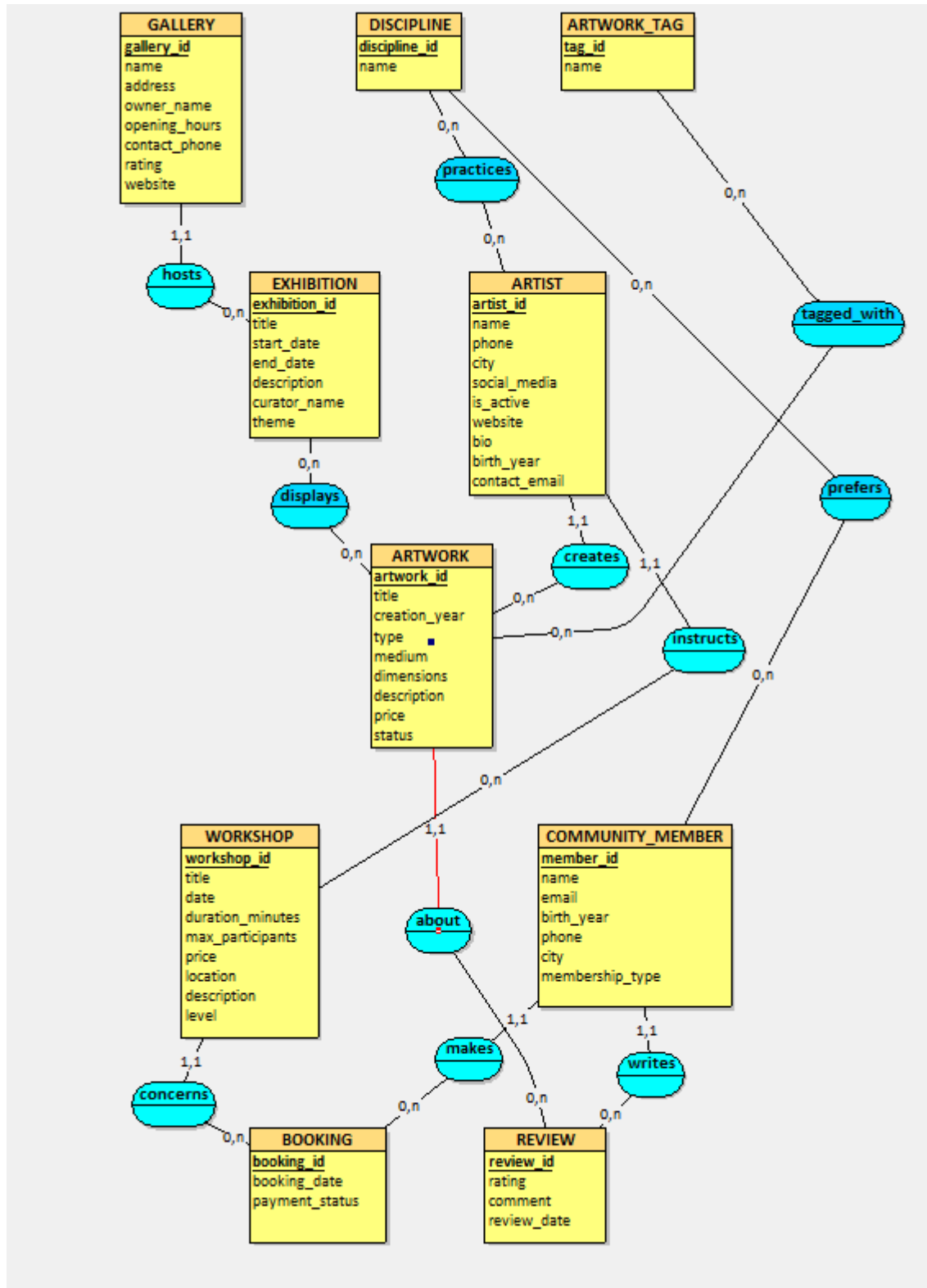


Class Diagram:



Step 3: Conceptual and Logical Modeling of the ArtConnect Database

Conceptual Data Model:



Logical Data Model:

EXHIBITION = (exhibition_id, title, start_date, end_date, description, curator_name, theme);
DISCIPLINE = (discipline_id, name);
ARTWORK_TAG = (tag_id, name);
BOOKING = (booking_id, booking_date, payment_status);
REVIEW = (review_id, rating, comment, review_date);
ARTWORK = (artwork_id, title, creation_year, type, medium, dimensions, description, price, status, #review_id);
GALLERY = (gallery_id, name, address, owner_name, opening_hours, contact_phone, rating, website, #exhibition_id);
WORKSHOP = (workshop_id, title, date, duration_minutes, max_participants, price, location, description, level, #booking_id);
COMMUNITY_MEMBER = (member_id, name, email, birth_year, phone, city, membership_type, #review_id, #booking_id);
ARTIST = (artist_id, name, phone, city, social_media, is_active, website, bio, birth_year, contact_email, #workshop_id, #artwork_id);
practices = (#artist_id, #discipline_id);
prefers = (#member_id, #discipline_id);
displays = (#artwork_id, #exhibition_id);
tagged_with = (#artwork_id, #tag_id);

Normalization Steps:

First Normal Form (1NF):

All tables satisfy 1NF because every column contains a single atomic value. For example, instead of storing disciplines as a comma-separated list inside the artist table, a separate discipline table was created alongside a junction table artist_discipline to link artists to their disciplines. Each cell contains exactly one value.

Second Normal Form (2NF):

All tables satisfy 2NF because every non-key attribute depends on the entire primary key. In junction tables such as artist_discipline, the composite primary key is (artist_id, discipline_id) and there are no other attributes, that way nothing can partially depend on only one part of the key.

Third Normal Form (3NF):

All tables satisfy 3NF because no non-key column depends on another non-key column. For example, the artwork table stores artist_id as a foreign key instead of duplicating artist_name or artist_city. This eliminates transitive dependency, that way, if an artist changes city, only the artist table needs updating.

Step 3: Database Implementation and Advanced Features

Prompt used to generate the data:

"Generate a MySQL INSERT script for a database called artconnect_db with the following tables: artist, artwork, gallery, exhibition, workshop, community_member, booking, review, discipline, artwork_tag, artist_discipline, member_discipline, exhibition_artwork, artwork_artwork_tag. Insert at least 5 artists, 8 artworks, 3 galleries, 4 exhibitions, 4 workshops, 5 community members, 6 bookings, 5 reviews, 5 disciplines, 5 artwork tags. Respect all foreign key relationships and make the data realistic for a local art community platform."

Step 4: Integrating the Database into the ArtConnect Java Application

Brief Description of the Chosen Architecture:

The ArtConnect application follows a four-layer architecture that strictly separates responsibilities between layers. Each layer communicates only with the layer directly below it.

Layer 1 — Presentation Layer (JavaFX Controllers)

This layer contains the JavaFX controllers that manage the graphical interface. Each controller corresponds to one tab of the application: ArtistController, ArtworkController, GalleryController, ExhibitionController, WorkshopController and CommunityController. Controllers receive user interactions (button clicks, form submissions) and delegate all data operations to the service layer. Controllers never communicate directly with the database.

Layer 2 — Service Layer (Business Logic)

This layer contains the service interfaces and their implementations. The JDBC implementations, JdbcArtistService, JdbcArtworkService, JdbcGalleryService, JdbcWorkshopService and JdbcCommunityService, receive requests from the controllers and delegate all database operations to the DAO layer. This layer is also where business logic such as search filtering is applied.

Layer 3 — DAO Layer (Data Access)

This layer is responsible for all communication with MySQL. The five JDBC DAO classes, JdbcArtistDao, JdbcArtworkDao, JdbcGalleryDao, JdbcWorkshopDao and JdbcCommunityMemberDao, implement the DAO interfaces and use PreparedStatement to safely execute SQL queries. All JDBC resources are closed automatically using try-with-resources.

Layer 4 — Database Layer (MySQL)

The MySQL database artconnect_db stores all persistent data across 14 tables. It also contains 3 views, 3 indexes, 3 triggers, 2 stored procedures, and 1 stored function that enforce business rules and optimize query performance independently of the Java application.

The ServiceProvider Switch

The class ServiceProvider acts as the central factory for all service instances. A single boolean constant USE_DATABASE controls whether the application uses the JDBC implementations connected to MySQL, or the original InMemory implementations for testing. Setting it to true activates the full database integration

Demo Video:

 Video_ArtConnect_App.mp4

GitHub Link:

<https://github.com/meissac/ArtConnectProject>